विशाल जाधव सरांचे

VJTech Academy

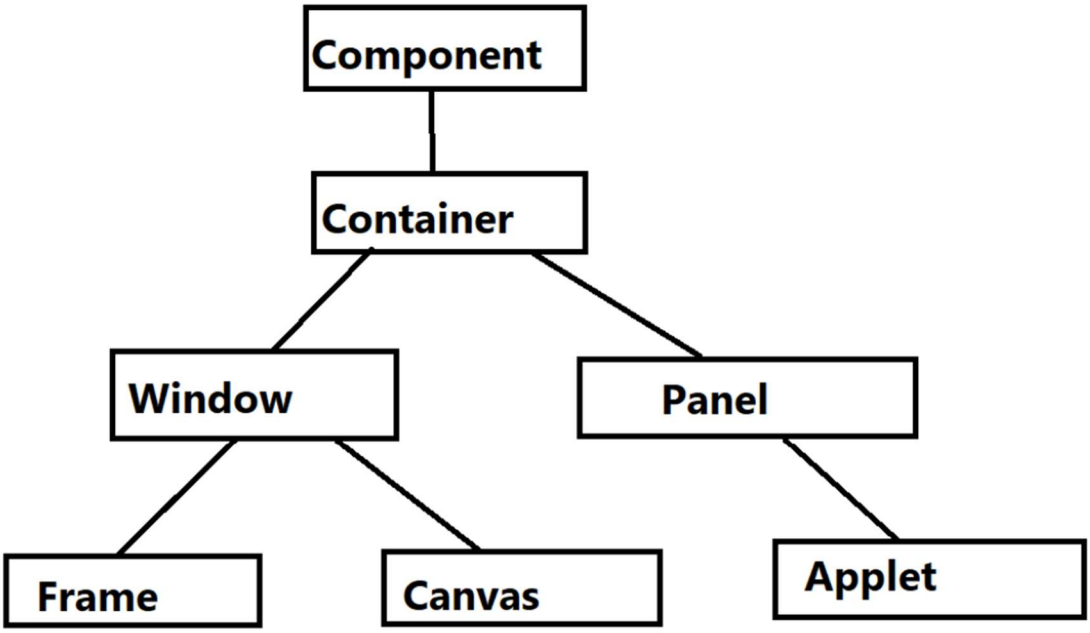
Inspiring Your Success...

UNIT-IV Event Handling using AWT and Swings Components

Syllabus

- 4.1 Component, container, window, frame, panel, use of AWT controls: labels, buttons, checkbox, checkbox group, textfield, textarea
- 4.2 Use of layout managers: flowLayout, borderLayout, gridLayout, gridBagLayout, menubars, menus, file dialog
- 4.3 Introduction to swing: Swing features, difference between AWT and Swing.
- 4.4 Swing components: Icons and Labels, TextField, ComboBox, Button, Checkbox, RadioButton
- 4.5 Advanced Swing Components: Tabbed Panes, Scroll Panes, Trees, Tables, Progress bar, tool tips
- 4.6 Introduction to Event Handling: The delegation Event Model: Event sources, Event listeners
- 4.7 Event classes: The action event class, the Item event class, the Key event class, the mouse event class, text event
- 4.8 Event listener interfaces: ActionListener, ItemListener, KeyListener, MouseListener, MouseMotion, TextListener

Component Class Hierarchy:



Component:

- In Java, the concept of a **Component** refers to a class that represents a graphical element (such as buttons, labels, or text fields) in a user interface.
- **Component class** is specifically used in **Abstract Window Toolkit (AWT)** and **Swing**, which are libraries for building graphical user interfaces (GUIs) in Java.
- The `java.awt.Component` class is the root class for most of the GUI elements in Java.
- It is part of the **AWT (Abstract Window Toolkit)** package but it is also inherited by Swing components
- The `Component` class is a direct subclass of `Object`.
- It is the superclass for many UI components such as `Button`, `TextField`, `Label`, `Panel`, etc.
- Other components like those in Swing (e.g., `JButton`, `JLabel`, `JTextField`) also extend from `Component`.
- This class also responsible for creating and managing events.
- The `Component` class is foundational in Java for creating graphical user interface elements.
- It provides essential methods for managing the behavior, appearance, and interactions of these elements within a container.

Methods:

- void setLocation(int x,int y)
- void setSize(int width,int height)
- void setBounds(int x,int y,int width,int height)
- void setLayout(Layoutmanager object)
- void setEnabled(boolean enabled)

Container:

- In AWT, the `Container` class is a subclass of `Component`.
- A container in AWT can hold components such as buttons, labels, text fields, and panels.
- Common AWT Containers
  - `Frame`: A top-level window that can contain other components.
  - `Panel`: A container that can hold a group of components.
  - `Dialog`: A pop-up window for interaction.
  - `Window`: A top-level window.

Methods:

- void add(Component comp) – Adds a component to the container.
- void remove(Component comp) – Removes a component from the container.
- void setLayout(LayoutManager mgr) – Sets the layout manager for the container.
- LayoutManager getLayout() – Returns the layout manager for the container.

Window:

- The `Window` class in AWT (Abstract Window Toolkit) is a top-level container.
- It is part of the `java.awt` package and is designed to provide a blank window
- By default, the `Window` class does not have a title bar or any border
- It can be used to create modal windows
- `Window` class is heavyweight because it depends upon the native OS Components for its implementation

Frame:

- Frame is a standard window which has title bar, menubar, minimize, maximize button and resizing corner.
- It is a subclass of the `Window` class.
- The `Frame` class is used for creating a window with a title.
- Components can be added to the frame, and window events can be handled using listeners.

Constructors:

1) Frame() - create window without title.
2) Frame(String title) - create window with title.

Methods:

1) void setVisible(Boolean flag) - Makes the frame visible or invisible. By default, it is invisible when created.
2) void setSize(int width, int height) - Sets the size of the frame.
3) void setTitle(String title) - Sets the title of the frame.

Example:

Program using default constructor i.e. Frame()

```
import java.awt.*;
class FrameDemo extends Frame
{
        public static void main(String args[])
        {
                FrameDemo f1=new FrameDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

Program using parameterized constructor i.e. Frame(String title)

```
import java.awt.*;
class FrameDemo extends Frame
{
	FrameDemo(String msg)
	{
		super(msg);
	}
	public static void main(String args[])
	{
		FrameDemo f1=new FrameDemo("VJTech Frame");
		f1.setVisible(true);
		f1.setSize(800,500);
	}
}
```

Panel:

- The `Panel` class is a container that is used to organize components in a group.
- It is a lightweight container, meaning it does not have a border or title bar.
- `Panel` can hold other components like buttons, text fields, etc.
- `Panel` is a direct subclass of `Container`
- It uses the **LayoutManager** for organizing components (like `FlowLayout`, `BorderLayout`, etc.).

Constructor:

1) Panel() - This constructor creates an empty panel

Label Control:

- In Java AWT, the `Label` class is a part of the `java.awt` package.
- It is used to create a non-editable text element that displays a single line of text.
- It can be used for displaying text.

Constructors:

1. **Label()** - This is the default constructor that creates an empty label.
2. **Label(String str)** - This constructor creates a label with the specified text.
3. **Label(String text, int alignment)** - This constructor creates a label with the specified text and alignment. The alignment can be one of the following:
   - `Label.LEFT`
   - `Label.CENTER`
   - `Label.RIGHT`

Methods:

1. **void setText(String text)** - Sets the text of the label.
2. **String getText()** - Gets the text of the label.
3. **void setAlignment(int alignment)** - Sets the alignment of the label's text
4. **int getAlignment()** - Returns the current alignment of the label's text

Example:

```
import java.awt.*;
class LabelDemo extends Frame
{
        LabelDemo()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                Label L1=new Label("India");
                Label L2=new Label("America");
                Label L3=new Label("Pune");
                add(L1);
                add(L2);
                add(L3);
```

```
        }
        public static void main(String args[])
        {
                LabelDemo f1=new LabelDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("Label Demo");
        }
}
```

Button Control:

- In Java AWT, **Button** is a UI component that allows users to interact with an application by clicking it.
- It is used to create push button.
- We can create button with or without label.

Constructor:

1. **Button()** - This constructor creates an empty button without any label.
2. **Button(String str)** - This constructor creates a button with the specified label.

Methods:

1. **void setLabel(String text)** - Sets the label on the button.
2. **String getLabel()** - Returns the label on the button.

Example:

```
import java.awt.*;
class ButtonDemo extends Frame
{
	ButtonDemo()
	{
		FlowLayout f1=new FlowLayout();
		setLayout(f1);
		Button b1=new Button("OK");
		Button b2=new Button("CANCEL");
		Button b3=new Button("RETRY");
		add(b1);
		add(b2);
		add(b3);
	}
	public static void main(String args[])
	{
		ButtonDemo bd=new ButtonDemo();
		bd.setVisible(true);
		bd.setSize(500,500);
		bd.setTitle("VJTech Academy");
	}
}
```

Checkbox Control:

- CheckBox is a rectangular box which has associated label.
- It states are ON or OFF.
- It is used to select multiple options among group.

Constructor:

1. **Checkbox()** - It will create checkbox without label.
2. **Checkbox(String label)** - It will create checkbox with label.
3. **Checkbox(String label, boolean state)** - Creates a checkbox with a label and a specified initial state (checked or unchecked).
4. **Checkbox(String label, CheckboxGroup group, boolean state)** - Creates a checkbox with a label, a specified group (for radio button functionality), and an initial state (checked or unchecked).

Methods:

1. **boolean getState()**- Returns the current state of the checkbox (either `true` for checked or `false` for unchecked).
2. **void setState(boolean state)** - Sets the checkbox's state. If `true`, the checkbox is checked; if `false`, the checkbox is unchecked.
3. **String getLabel()** - Returns the label of the checkbox.
4. **void setLabel(String label)** - Sets a new label for the checkbox.

Example:

```
import java.awt.*;
class CheckboxDemo extends Frame
{
        CheckboxDemo()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                Checkbox c1=new Checkbox("C Lang");
                Checkbox c2=new Checkbox("C++ Lang",true);
                Checkbox c3=new Checkbox("Java Lang");
                add(c1);
                add(c2);
```

```
                add(c3);
        }
        public static void main(String args[])
        {
                CheckboxDemo cbd=new CheckboxDemo();
                cbd.setVisible(true);
                cbd.setSize(500,500);
                cbd.setTitle("Checkbox Demo");
        }
}
```

CheckboxGroup:

- It will convert checkbox into radio-button
- **Constructor:** CheckboxGroup();
- **Method:** Checkbox getSelectedCheckbox();
- **Example:**

```
import java.awt.*;
class CheckboxDemo extends Frame
{
        CheckboxDemo()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                CheckboxGroup cbg=new CheckboxGroup();
                Checkbox cb1=new Checkbox("Male",cbg,false);
                Checkbox cb2=new Checkbox("Female",cbg,true);
                add(cb1);
                add(cb2);
        }
        public static void main(String args[])
        {
                CheckboxDemo cbd=new CheckboxDemo();
                cbd.setVisible(true);
                cbd.setSize(500,500);
                cbd.setTitle("Checkbox Demo");
        }
}
```

TextField Control:

- The `TextField` control is a single-line text input field that allows users to enter and edit text.
- It is a part of the `java.awt` package and provides various constructors and methods to manipulate text input.
- This control used to accept input from the user.

Constructor:

1. **TextField()** - Creates an empty text field with the default width.
2. **TextField(String text)** - Creates a text field with the specified initial text.
3. **TextField(int columns)** - Creates a text field with the specified number of columns, which determines its width.
4. **TextField(String text, int columns)** - Creates a text field with the specified initial text and the number of columns.

Methods:

1. **void setText(String text)** - Sets the text of the text field to the specified string.
2. **String getText()** - Returns the current text in the text field as a string.
3. **void setEditable(boolean editable)** - Enables or disables editing in the text field. If set to `false`, the text field becomes read-only.
4. **Boolean isEditable()** - Returns `true` if the text field is editable, and `false` otherwise.
5. **void setEchoChar(char x)** - Sets the echo character for the text field
6. **char getEchoChar()** - Returns the current echo character for the text field.
7. **void select(int selectionStart, int selectionEnd)** - Selects a specific range of text between the given start and end positions

Example:

```
import java.awt.*;
class TextFieldDemo extends Frame
{
	TextFieldDemo()
	{
		FlowLayout f1=new FlowLayout();
		setLayout(f1);
		setBackground(Color.green);
		Label L1=new Label("Enter Your Name:");
		TextField tf1=new TextField(20);
		Button b1=new Button("Submit");
		add(L1);
		add(tf1);
		add(b1);
	}
	public static void main(String args[])
	{
		TextFieldDemo f1=new TextFieldDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("TextField Demo");
	}
}
```

TextArea Control:

- The `TextField` control is a multiple lines text input field that allows users to enter and edit text.
- It is a part of the `java.awt` package
- This control used to accept multiple lines of inputs from the user.

Constructor:

1. **TextArea()**- This constructor creates a new empty `TextArea` with no text and no initial number of rows or columns.
2. **TextArea(String text)** - Creates a new `TextArea` with the specified initial text.
3. **TextArea(String text, int rows, int columns)** - Creates a new TextArea with the specified initial text, number of rows, and columns.
4. **TextArea(String text, int rows, int columns, int scrollBars)** - This constructor creates a new `TextArea` with the specified initial text, number of rows, columns, and a `scrollBars` setting. `scrollBars` can be:
   - `TextArea.SCROLLBARS_NONE` : No scrollbars.
   - `TextArea.SCROLLBARS_VERTICAL_ONLY` : Only vertical scrollbar.
   - `TextArea.SCROLLBARS_HORIZONTAL_ONLY` : Only horizontal scrollbar.
   - `TextArea.SCROLLBARS_BOTH` : Both vertical and horizontal scrollbars.

Methods:

1. **void setText(String text)** - Sets the specified text in the `TextArea`.
2. **String getText()** - Returns the current text in the `TextArea` as a `String`..
3. **void append(String text)** - Appends the specified text to the end of the current text.
4. **void insert(String text, int position)** - Inserts the specified text at the given position in the current text.
5. **void setEditable(boolean editable)** - Enables or disables editing in the textarea. If set to `false`, the text area becomes read-only.
6. **Boolean isEditable()** - Returns `true` if the text area is editable, and `false` otherwise.

Example:

```
import java.awt.*;
class TextAreaDemo extends Frame
{
	TextAreaDemo()
	{
		FlowLayout f1=new FlowLayout();
		setLayout(f1);
		TextArea ta1=new TextArea("Write your feedback",15,25);
		add(ta1);
	}
	public static void main(String args[])
	{
		TextAreaDemo tad=new TextAreaDemo();
		tad.setVisible(true);
		tad.setSize(500,500);
		tad.setTitle("TextArea Demo");
	}
}
```

Working with Menubar, Menus:

- A menubar is most useful component.
- Menubar present in top level of window.
- Three different classes (Menu, MenuBar, MenuItem) - MenuBar contains Menus and Menus contains MenuItems.

MenuBar Class:

- It is a container that holds one or more `Menu` components.
- A `MenuBar` is usually added to a frame (such as `Frame` or `Window`) and typically appears at the top of the window.
- First, we will create MenuBar class object.
- **Constructor**: MenuBar();
- **Method:** void setMenuBar(MenuBar object)

Menu Class:

- Menu is a predefined class and it is used to create Menus.
- It represents a single drop-down menu, containing one or more `MenuItem` components.
- A `Menu` can also contain submenus
- **Constructor:**

  1) Menu() - it will create empty Menu.

  2) Menu(String str) - it will create Menu with name specified in str.

  3) Menu(String str,boolean flag) - it will create Menu with name specified in str, if flag is true then pop-up menu can be removed and allowed to float free in Menubar. And if it false then it will remain attach to the Menubar.

MenuItem Class:

- A `MenuItem` represents an individual item in a menu.
- Clicking a menu item performs an action, typically associated with a specific function (like opening a file or saving).
- Menu items can be added to a `Menu`
- **Constructor**:

  1) MenuItem();

2) MenuItem(String str) - it will create MenuItem with name.

3) MenuItem(String str,MenuShortcut key)

###CheckboxMenuItem Class

Constructor

1) CheckboxMenuItem()
2) CheckboxMenuItem(String str);
3) CheckboxMenuItem(String Str, boolean flag);

Methods:

1) void setEnabled(boolean flag)

2) boolean isEnable()

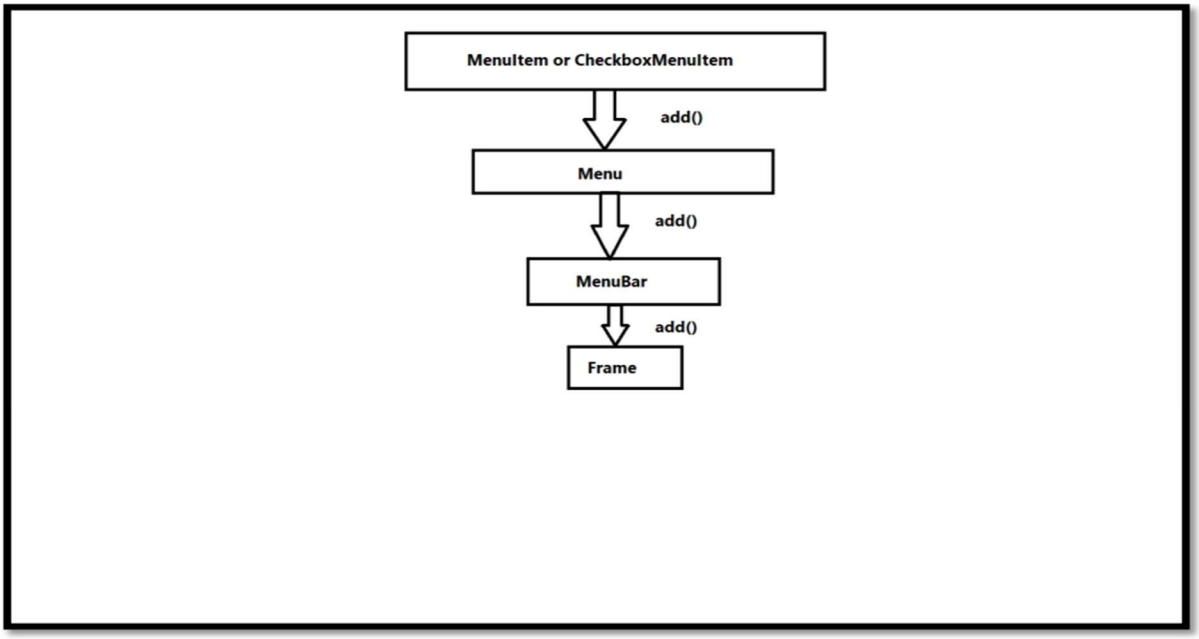
3) void setLabel(String str)

4) String getLabel()

5) void setState(boolean flag)

6) boolean getState()

Diagram:-



Example:

```
import java.awt.*;
import java.awt.event.*;
class Notepad extends Frame
{
    Notepad()
    {
        MenuBar mbr=new MenuBar();

        MenuShortcut msc1=new MenuShortcut(KeyEvent.VK_N);
        MenuShortcut msc2=new MenuShortcut(KeyEvent.VK_O);

        Menu m1=new Menu("File");
        MenuItem m11=new MenuItem("New",msc1);
        MenuItem m12=new MenuItem("Open",msc2);
        MenuItem m13=new MenuItem("Save");
        MenuItem m14=new MenuItem("Save As");

        m1.add(m11);
        m1.add(m12);
        m1.add(m13);
        m1.add(m14);

        Menu m2=new Menu("Edit");
        MenuItem m21=new MenuItem("Cut");
        MenuItem m22=new MenuItem("Copy");
        MenuItem m23=new MenuItem("Paste");
        MenuItem m24=new MenuItem("Undo");
        m2.add(m21);
        m2.add(m22);
        m2.add(m23);
        m2.add(m24);

        Menu m3=new Menu("Search");
        MenuItem m31=new MenuItem("Find");
        MenuItem m32=new MenuItem("Find Next");
        MenuItem m33=new MenuItem("Find Previous");
        MenuItem m34=new MenuItem("Replace");
        m3.add(m31);
        m3.add(m32);
        m3.add(m33);
        m3.add(m34);

        Menu m4=new Menu("View");
        MenuItem m41=new MenuItem("Always on Top");
        MenuItem m42=new MenuItem("Post IT");
```

```
            Menu m43=new Menu("Zoom");
            MenuItem m431=new MenuItem("Zoom In");
            MenuItem m432=new MenuItem("Zoom Out");
            m43.add(m431);
            m43.add(m432);

            CheckboxMenuItem m44=new CheckboxMenuItem("Word wrap",true);

            m4.add(m41);
            m4.add(m42);
            m4.add(m43);
            m4.add(m44);

            mbr.add(m1);
            mbr.add(m2);
            mbr.add(m3);
            mbr.add(m4);

            setMenuBar(mbr);
    }
    public static void main(String args[])
    {
            Notepad n1=new Notepad();
            n1.setVisible(true);
            n1.setSize(500,500);
            n1.setTitle("VJTech Notepad");
    }
}
```

FileDialog:

- In Java, the FileDialog class is part of the AWT package and is used to provide a dialog box that allows users to choose files or directories.
- It is typically used for opening or saving files.
- This class is used to display file dialog box.
- This is standard file dialogbox which is provided the operating system.
- FileDialog is a child window.

Constructor:

1. **FileDialog(Frame parent, String title)** - Creates a new `FileDialog` with the specified parent frame and title.
2. **FileDialog(Frame parent, String title, int mode)** - Creates a new `FileDialog` with the specified parent frame, title, and mode. The mode defines whether the dialog will be used for opening or saving files.
3. **FileDialog(Frame parent)** - Creates a new `FileDialog` without specifying a title. The mode is by default set to `LOAD` (open).

Methods:

1. **void setTitle(String title)** - Sets the title of the dialog window.
2. **String getTitle()** - Returns the title of the dialog.
3. **void setFile(String filename) -** The name of the file to be shown in the dialog box.
4. **String getFile()** - Returns the name of the file selected by the user
5. **void setDirectory(String directory) -** Sets the initial directory shown in the file selection area.
6. **String getDirectory()** - Returns the path of the directory that was displayed when the dialog was opened
7. **void setMode(int mode)** - Sets the mode of the dialog.
   - `FileDialog.LOAD` – For opening files.
   - `FileDialog.SAVE`– For saving files.
8. **int getMode() -** Returns the current mode of the dialog.

Example:

```
import java.awt.*;
class FileDialogDemo extends Frame
{
	public static void main(String args[])
	{
		FileDialogDemo f1=new FileDialogDemo();
		f1.setVisible(true);
		f1.setSize(900,900);
		f1.setTitle("Parent Frame");
		FileDialog fd=new FileDialog(f1,"Save",FileDialog.SAVE);
		fd.setVisible(true);
		fd.setSize(500,500);
	}
}
```

Use of Layout Managers:

- In AWT, Layout Managers are used to automatically arrange components within a container.
- They control how components are arranged when the container is resized or when the components are added or removed.
- Without layout managers, you would have to manually set the position and size of each component, which would be inefficient and difficult to manage.
- There are several types of layout managers in AWT.

1. FlowLayout
2. BorderLayout
3. GridLayout
4. GridBagLayout

FlowLayout Manager:

- The `FlowLayout` manager in Java is part of the **AWT** and **Swing**.
- It arranges components in a **left-to-right flow**, much like lines of text in a paragraph. When the line is filled, it goes to the next line.
- By default it will add components in center of window.

Constructor:

1. **FlowLayout()** - Creates a flow layout with centered alignment.
2. **FlowLayout(int align)** - Creates a flow layout with the specified alignment: `FlowLayout.LEFT`, `FlowLayout.CENTER`, or `FlowLayout.RIGHT`.
3. **FlowLayout(int align, int hgap, int vgap)** - Specifies alignment, horizontal gap, and vertical gap between components..

Example:

```
import java.awt.*;
class FlowLayoutDemo extends Frame
{
        FlowLayoutDemo()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                Button b1=new Button("OK");
                Button b2=new Button("CANCEL");
```

```
                Button b3=new Button("RETRY");
                add(b1);
                add(b2);
                add(b3);
        }
        public static void main(String args[])
        {
                FlowLayoutDemo f1=new FlowLayoutDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

❖ BorderLayout Manager:

- The `BorderLayout` manager is one of the layout managers in AWT and Swing.
- It's used to arrange components in **five regions**: `NORTH`, `SOUTH`, `EAST`, `WEST`, and `CENTER`.
- This class present under java.awt package.
- Only one component can be added to each region. The `CENTER` region takes up whatever space is left after the others are allocated.
- It divides the container into five areas:
  - `BorderLayout.NORTH`
  - `BorderLayout.SOUTH`
  - `BorderLayout.EAST`
  - `BorderLayout.WEST`
  - `BorderLayout.CENTER`

Constructor:

1. **BorderLayout()** - Creates a border layout with horizontal and vertical gaps set to 0.
2. **BorderLayout(int hgap, int vgap)** - Creates a border layout with specified horizontal and vertical gaps.

Method:

1. **void add(Component object, int region)** - Component object that you want to add and region values - BorderLayout.EAST, BorderLayout.WEST, BorderLayout.SOUTH, BorderLayout.NORTH, BorderLayout.CENTER.

Example:

```
import java.awt.*;
class BorderLayoutDemo extends Frame
{
	BorderLayoutDemo()
	{
		BorderLayout f1=new BorderLayout(10,10);
		setLayout(f1);
		Button b1=new Button("EAST Button");
		Button b2=new Button("WEST Button");
		Button b3=new Button("SOUTH Button");
		Button b4=new Button("NORTH Button");
		TextArea ta1=new TextArea("Enter text here");
		add(b1,BorderLayout.EAST);
		add(b2,BorderLayout.WEST);
		add(b3,BorderLayout.SOUTH);
		add(b4,BorderLayout.NORTH);
		add(ta1,BorderLayout.CENTER);
	}
	public static void main(String args[])
	{
		BorderLayoutDemo f1=new BorderLayoutDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("VJTech Frame");
	}
}
```

❖ GridLayout Manager:

- The `GridLayout` manager in AWT is used to arrange components in a rectangular grid.
- When using `GridLayout`, all components are given equal size and are placed in the cells of the grid in order left to right, top to bottom.
- GridLayout manager is used to display the components in two dimensional grids.

Constructor:

1. **GridLayout()** - it will display the components in single row and single column.
2. **GridLayout(int rows, int columns)** - you can display components in no of rows and no of columns.
3. **GridLayout(int rows,int columns,int horizonal_gap,int vertical_gap)** - Adds horizontal and vertical gaps between components

Example:

```
import java.awt.*;
class GridLayoutDemo extends Frame
{
	GridLayoutDemo()
	{
		GridLayout g1=new GridLayout(5,5);
		setLayout(g1);
		for(int i=1;i<=25;i++)
		{
			add(new Button("Button "+i));
		}
	}
	public static void main(String args[])
	{
		GridLayoutDemo f1=new GridLayoutDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("VJTech Frame");
	}
}
```

❖ GridBagLayout Manager:

- `GridBagLayout` is one of the most flexible and complex layout managers in.
- It aligns components vertically, horizontally, or along their baseline without requiring components to be the same size.
- It uses a grid of cells, but components can span multiple cells.
- `GridBagConstraints` is a helper class that holds layout information for each component.

Constructor:

1. **GridBagLayout()** - There's only one constructor: the default constructor, which creates a new `GridBagLayout` object.

Methods:

gridx, gridy	Position in the grid (column, row).
gridwidth, gridheight	Number of cells a component uses.
weightx, weighty	Space distribution when the window is resized.
ipadx, ipady	Internal padding (space inside component).
fill	Whether to resize the component (NONE, HORIZONTAL, VERTICAL, BOTH).

Example:

```
import javax.swing.*;
import java.awt.*;
class GridBagLayoutDemo extends JFrame
{
        GridBagLayoutDemo()
        {
                GridBagLayout g1=new GridBagLayout();
                setLayout(g1);

                GridBagConstraints gbc = new GridBagConstraints();

                JButton b1 = new JButton("Button1");
                gbc.gridx = 0;
                gbc.gridy = 0;
                add(b1,gbc);

                JButton b2 = new JButton("Button2");
                gbc.gridx = 1;
                gbc.gridy = 0;
```

```
                add(b2,gbc);

                JButton b3 = new JButton("Button3");
                gbc.gridx = 0;
                gbc.gridy = 1;
                gbc.gridwidth = 2; // spans two columns
                gbc.fill = GridBagConstraints.HORIZONTAL;
                add(b3,gbc);
        }
        public static void main(String[] args)
        {
                GridBagLayoutDemo f1=new GridBagLayoutDemo();
                f1.setSize(500,500);
                f1.setVisible(true);
                f1.setTitle("GridBagLayout Demo");
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

Introduction to Swing (Java Swing):

- Swing is a part of Java Foundation Classes (JFC) used to create Graphical User Interfaces (GUIs) in Java.
- It provides a rich set of components such as buttons, text fields, tables, trees, etc., and is built on top of AWT.
- Swing was introduced to overcome the limitations of AWT and provides more powerful and flexible GUI components.

✓ Features of Swing

1. **Platform Independent**

   Swing components are written in Java and do not rely on the native system GUI.
2. **Lightweight Components**

   Unlike AWT, Swing components do not use native OS resources, making them lightweight.
3. **Pluggable Look and Feel**

   You can change the appearance of Swing applications to mimic different OS themes (e.g., Windows, GTK, Metal).
4. **Rich Controls**

   Swing provides advanced components like tables (`JTable`), trees (`JTree`), sliders, tabbed panes, etc.
5. **MVC Architecture**

   Swing follows the **Model-View-Controller (MVC)** architecture, which separates data from the user interface.

Difference Between AWT and Swing

Difference between AWT and Swing

AWT	Swing
1) AWT stands for Abstract Window Toolkit.	1) Swing is also called as JFC(Java Foundation classes)
2) Requires to import **java.awt** package	2) Requires to import **javax.swing** package
3) It is platform dependent code	3) It is platform independent code
4) AWT components are heavy weight.	4) Swing components are light weight.
5) AWT is a huge collection of classes and interfaces	5) Swing is a bigger collection of classes and interfaces than AWT.
6) AWT provides less features and components than Swing	6) Swing has got variety of components and more features than AWT.
7) Lookwise AWT components are less effective than swing.	7) Lookwise swing components are more effective than AWT.
8) Examples: Button, Label, TextField, etc	8) Examples: JButton, JLabel, JTextField, etc

JFrame:

- JFrame is a standard window which has title bar, menubar, minimize, maximize button and resizing corner.
- The `JFrame` class is used for creating a window with a title.
- It is present under javax.swing package

Constructors:

1) JFrame() - create window without title.
2) JFrame(String title) - create window with title.

Methods:

1) void setVisible(Boolean flag) - Makes the frame visible or invisible. By default, it is invisible when created.
2) void setSize(int width, int height) - Sets the size of the frame.
3) void setTitle(String title) - Sets the title of the frame.
4) void setDefaultCloseOperation(int operation) – Defines behavior when the window is closed

operation:-

JFrame.EXIT_ON_CLOSE

JFrame.HIDE_ON_CLOSE

JFrame.DO_NOTHING_ON_CLOSE

Above given three static constants of WindowConstants interface.

Example:

Program using default constructor i.e. JFrame()

```
import javax.swing.*;
class JFrameDemo extends JFrame
{
	public static void main(String args[])
	{
		JFrameDemo f1=new JFrameDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("VJTech Frame");
		f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
	}
}
```

Program using parameterized constructor i.e. JFrame(String title)

```
import javax.swing.*;
class JFrameDemo1 extends JFrame
{
	JFrameDemo1(String msg)
	{
		super(msg);
	}
	public static void main(String args[])
	{
		JFrameDemo1 f1=new JFrameDemo1("VJTech Frame");
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
	}
}
```

ImageIcon Class:

- If we want to apply image on any supported components, then we can use ImageIcon class.
- It is present under javax.swing package

Constructors:

1) ImageIcon(String filename)
2) ImageIcon(URL url)
3) ImageIcon()

Methods:

1) Image getImage() - Returns the Image object
2) int getIconWidth() - Width of the icon
3) int getIconHeight() - Height of the icon
4) void setImage(Image image) - Sets the image
5) void paintIcon(Component c, Graphics g, int x, int y) - Draws the icon at specified position on a component.

Example:

```
import javax.swing.*;
import java.awt.*;
class JLabelDemo extends JFrame
{
        JLabelDemo()
        {
                Container c=getContentPane();
                FlowLayout f1=new FlowLayout();
                c.setLayout(f1);
                ImageIcon ii=new ImageIcon("vjtech.png");
                JLabel L1=new JLabel("VJTech Academy",ii, JLabel.LEFT);
                c.add(L1);
        }
        public static void main(String args[])
        {
                JLabelDemo f1=new JLabelDemo();
```

```
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JLabel Class:

- The `JLabel` class is part of the `javax.swing` package and is used to display a short string or an image on the screen.
- It is a non-editable component that can be used for presenting text or images in a GUI (Graphical User Interface).
- It is commonly used to display text.
- It can also display images using an `ImageIcon`.
- You can align the text or image within the label

Constructors:

1) `JLabel(String text)`: Creates a label with the specified text.
2) `JLabel(Icon icon)`: Creates a label with the specified icon.
3) JLabel(String str, ImageIcon ii, int alignment) - alignment -> JLabel.RIGHT, JLabel.LEFT, JLabel.CENTER, JLabel.LEADING, JLabel.TRAILING
4) JLabel()

Methods:

1) void setText(String str)
2) String getText()
3) void setAlignment(int alignment)
4) int getAlignment()
5) void setIcon(ImageIcon ii)
6) ImageIcon getIcon()

Example:

```
import javax.swing.*;
import java.awt.*;
class JLabelDemo extends JFrame
{
        JLabelDemo()
        {
                Container c=getContentPane();
```

```
                FlowLayout f1=new FlowLayout();
                c.setLayout(f1);
                ImageIcon ii=new ImageIcon("vjtech.png");
                JLabel L1=new JLabel("VJTech Academy",ii, JLabel.LEFT);
                c.add(L1);
        }
        public static void main(String args[])
        {
                JLabelDemo f1=new JLabelDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JTextField Class:

- The `JTextField` class is a part of the `javax.swing` package and is used to create a single-line text input field for user interaction.
- It allows the user to input text or numbers.
- This is most popular component of swing.
- You can use it to build forms, search bars, or any other input field in your GUI application.

Constructors:

1) JTextField(): Default constructor. Creates an empty text field.
2) JTextField(int columns): Creates a text field with the specified number of columns.
3) JTextField(String text): Creates a text field with the specified initial text.
4) JTextField(String text, int columns): Creates a text field with the specified initial text and the specified number of columns.

Methods:

1) void setText(String str)
2) String getText()
3) void setColumns(int columns)
4) void setEditable(boolean editable)

Example:

```
import javax.swing.*;
import java.awt.*;
class JTextFieldDemo extends JFrame
{
	JTextFieldDemo()
	{
		Container c=getContentPane();
		FlowLayout f1=new FlowLayout();
		c.setLayout(f1);
		JLabel L1=new JLabel("Enter Your Name:");
		JTextField tf1=new JTextField(20);
		JButton b1=new JButton("Submit");
		c.add(L1);c.add(tf1);
		c.add(b1);
```

```
        }
        public static void main(String args[])
        {
                JTextFieldDemo f1=new JTextFieldDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JComboBox Class:

- Swing provides JComboBox class that creates a combo box.
- It will create drop down list (pop-up list).
- JComboBox is a predefined class.
- We can select only one item from the given list.
- When user click on combo box control then list of items are displayed and user can select one of the item among the list.

Constructors:

1) JComboBox()
2) JComboBox(Vector obj)

Methods:

1) void addItem(Object obj)
2) void removeItem(Object obj)
3) void removeAllItems()

Example:

```
import javax.swing.*;
import java.awt.*;
class JComboBoxDemo extends JFrame
{
	JComboBoxDemo()
	{
		Container c=getContentPane();
		FlowLayout f1=new FlowLayout();
		c.setLayout(f1);
		JComboBox jcb=new JComboBox();
		jcb.addItem("C Lang");
		jcb.addItem("C++ Lang");
		jcb.addItem("Java Lang");
		jcb.addItem("Python Lang");
		jcb.addItem("Android Lang");
		c.add(jcb);
	}
```

```
        public static void main(String args[])
        {
                JComboBoxDemo f1=new JComboBoxDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JButton Class:

- It provides Push Button.
- We can set Label or Image on Button.
- JComboBox is a predefined class which is present under javax.swing package.
- It represents a push button that can trigger an action when clicked

Constructors:

1) JButton(String str)

2) JButton(ImageIcon ii)

3) JButton(String str,ImageIcon ii);

4) JButton();

Methods:

1) void setText(String text)

2) String getText()

3) void setIcon(Icon icon)

4) Icon getIcon()

Example:

```
import javax.swing.*;
import java.awt.*;
class JButtonDemo extends JFrame
{
        JButtonDemo()
        {
                Container c=getContentPane();
                FlowLayout f1=new FlowLayout();
                c.setLayout(f1);
                JButton b1=new JButton("OK");
                JButton b2=new JButton("Cancel");
                JButton b3=new JButton("Retry");
                c.add(b1);
                c.add(b2);
                c.add(b3);
```

```
        }
        public static void main(String args[])
        {
                JButtonDemo f1=new JButtonDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JCheckBox Class:

- It will creates checkbox which has rectangular box and its associated label.
- JCheckBox class is a predefined class which is present under javax.swing package.
- JCheckBox class is derived from JToggleButton class.
- JCheckBox has two states ON and OFF.
- We can set Images to the checkbox.

Constructors:

1) JCheckBox(String str)

2) JCheckBox(String str,boolean state)

3) JCheckBox(ImageIcon ii)

4) JCheckBox(ImageIcon ii, boolean state)

5) JCheckBox(String str,ImageIcon ii)

6) JCheckBox(String str,ImageIcon ii,boolean state)

Methods:

1) void setText(String text)

2) String getText()

3) void setIcon(Icon icon)

4) Icon getIcon()

5) boolean getState()

6) boolean isSelected();

Example:

```
import javax.swing.*;
import java.awt.*;
class JCheckBoxDemo extends JFrame
{
        JCheckBoxDemo()
        {
                Container c=getContentPane();
```

```
                FlowLayout f1=new FlowLayout();
                c.setLayout(f1);
                JCheckBox c1=new JCheckBox("India",true);
                JCheckBox c2=new JCheckBox("UK");
                JCheckBox c3=new JCheckBox("America");
                c.add(c1);
                c.add(c2);
                c.add(c3);
        }
        public static void main(String args[])
        {
                JCheckBoxDemo f1=new JCheckBoxDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JRadioButton Class:

- It will create radio button.
- It is used to select one option from multiple options.
- It is widely used in exam systems or quiz.
- JRadioButton class is derived from JToggleButton class.
- It is present under javax.swing package.
- We can set image to the radio button.
- If user selected radio button, then automatically deselected previous radio button.
- For this you need to create ButtonGroup class object and use add methods to add all radio button in this group.

Constructors:

1) JRadioButton(String str)
2) JRadioButton(String str, boolean state)
3) JRadioButton(ImageIcon ii)
4) JRadioButton(ImageIcon ii, boolean state)
5) JRadioButton(String str, ImageIcon ii)
6) JRadioButton(String str, ImageIcon ii, boolean state)

Methods:

1) void setText(String text)
2) String getText()
3) void setIcon(Icon icon)
4) Icon getIcon()
5) boolean isSelected();

Example:

```
import javax.swing.*;
import java.awt.*;
class JRadioButtonDemo extends JFrame
{
        JRadioButtonDemo()
        {
```

```
                Container c=getContentPane();
                FlowLayout f1=new FlowLayout();
                c.setLayout(f1);
                JRadioButton r1=new JRadioButton("Male");
                JRadioButton r2=new JRadioButton("Female");
                JRadioButton r3=new JRadioButton("Other");
                ButtonGroup bg=new ButtonGroup();
                bg.add(r1);
                bg.add(r2);
                bg.add(r3);

                c.add(r1);
                c.add(r2);
                c.add(r3);
        }
        public static void main(String args[])
        {
                JRadioButtonDemo f1=new JRadioButtonDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
                f1.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        }
}
```

JTabbedPane Class:

- JTabbedPane is a predefined class which is present under javax.swing package .
- It will create multiple tabs and each tab has its own title and related components.
- We can see only one tab at a time. We cannot open multiple tabs at same time.
- We use JTabbedPane for providing configuration related options.
- *** **Procedure**:

  1) Define required no of Panel classes which contains different components.

  2) Create JTabbedPane object

  3) Use addTab() method to add tabs into JTabbedPane (title and component object)

  4) Add JTabbedPane object into Container. .

Constructor:

1) JTabbedPane()

Methods:

1) void addTab(String title, Component obj)

Example:

```
import javax.swing.*;
import java.awt.*;
class JPanel1 extends JPanel
{
        JPanel1()
        {
                JButton b1=new JButton("OK");
                JButton b2=new JButton("CANCEL");
                JButton b3=new JButton("RETRY");
                add(b1);
                add(b2);
                add(b3);
        }
}
class JPanel2 extends JPanel
{
        JPanel2()
        {
                JCheckBox b1=new JCheckBox("C Lang");
                JCheckBox b2=new JCheckBox("C++ Lang");
                JCheckBox b3=new JCheckBox("Java Lang");
```

```
                add(b1);
                add(b2);
                add(b3);
        }
}
class JPanel3 extends JPanel
{
        JPanel3()
        {
                JComboBox jcb=new JComboBox();
                jcb.addItem("India");
                jcb.addItem("UK");
                jcb.addItem("US");
                add(jcb);
        }
}
class JTabbedPaneDemo extends JFrame
{
        JTabbedPaneDemo()
        {
                Container c=getContentPane();
                JTabbedPane jtp=new JTabbedPane();

                JPanel1 p1=new JPanel1();
                JPanel2 p2=new JPanel2();
                JPanel3 p3=new JPanel3();

                jtp.addTab("Tab1",p1);
                jtp.addTab("Tab2",p2);
                jtp.addTab("Tab3",p3);

                c.add(jtp);
        }
        public static void main(String args[])
        {
                JTabbedPaneDemo f1=new JTabbedPaneDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

JScrollPane Class:

- JScrollPane is a predefined class which is present under javax.swing package .
- JScrollPane class provides rectangular scrollable area in which components can be viewed.
- The `JScrollPane` class in Java Swing is used to provide scrollable view for another component.
- If the component is too large to fit in the visible area, scroll bars will appear automatically.

Constructor:

1) JScrollPane(Component obj)

2) JScrollPane(int vertical, int horizontal)

3) JScrollPane(Component obj, int vertical, int horizontal)

Where: vertical, horizontal=> this is integer constants (defined in ScrollPaneConstants interface) which defines vertical and horizontal scrollbar.

HORIZONTAL_SCROLLBAR_ALWAYS

VERTICAL_SCROLLBAR_ALWAYS

HORIZONTAL_SCROLLBAR_AS_NEEDED

VERTICAL_SCROLLBAR_AS_NEEDED

Example:

```
import javax.swing.*;
import java.awt.*;
class JScrollPaneDemo extends JFrame
{
        JScrollPaneDemo()
        {
                Container c=getContentPane();
                JTextArea ta1=new JTextArea("Good Morning Friends",20,20);
                int v=ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS;
                int h=ScrollPaneConstants.HORIZONTAL_SCROLLBAR_ALWAYS;
                JScrollPane jsp=new JScrollPane(ta1,v,h);
                c.add(jsp);
        }
        public static void main(String args[])
        {
```

```
                JScrollPaneDemo f1=new JScrollPaneDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

JTree Class:

- JTree is a predefined class which is present under javax.swing package. .
- Hierarchical view of data is represented by using JTree.
- This component allows us to expand or collapse the individual subtree.
- JTree will derived properties of JComponent.
- *** **Procedure**:

1) Create JTree Object
2) Create JscrollPane
3) Add tree to the scrollpane.
4) Add ScrollPane to the container.

Constructors:

1) JTree(Hashtable ht)
2) JTree(Object obj[])
3) JTree(TreeNode tn)
4) JTree(Vector v)

DefaultMutableTreeNode class implements MutableTreeNode interface.

1) DefaultMutableTreeNode(Object obj)

Method:

1) void add(MutableTreeNode child);

Example:

```
import javax.swing.*;
import javax.swing.tree.*;
import java.awt.*;
class JTreeDemo extends JFrame
{
        JTreeDemo()
        {
                Container c=getContentPane();
                BorderLayout b1=new BorderLayout();
                c.setLayout(b1);
```

```
                DefaultMutableTreeNode lang=new DefaultMutableTreeNode("Language");

                DefaultMutableTreeNode pop=new DefaultMutableTreeNode("POP");
                DefaultMutableTreeNode p1=new DefaultMutableTreeNode("C Lang");
                DefaultMutableTreeNode p2=new DefaultMutableTreeNode("Fortran");
                DefaultMutableTreeNode p3=new DefaultMutableTreeNode("Cobol");
                DefaultMutableTreeNode p4=new DefaultMutableTreeNode("Pascal");
                pop.add(p1);
                pop.add(p2);
                pop.add(p3);
                pop.add(p4);

                DefaultMutableTreeNode oop=new DefaultMutableTreeNode("OOP");
                DefaultMutableTreeNode o1=new DefaultMutableTreeNode("C++");
                DefaultMutableTreeNode o2=new DefaultMutableTreeNode("Java");
                DefaultMutableTreeNode o3=new DefaultMutableTreeNode("Python");
                DefaultMutableTreeNode o4=new DefaultMutableTreeNode("C#");
                oop.add(o1);
                oop.add(o2);
                oop.add(o3);
                oop.add(o4);

                lang.add(pop);
                lang.add(oop);

                JTree jt=new JTree(lang);

                int v=ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS;
                int h=ScrollPaneConstants.HORIZONTAL_SCROLLBAR_ALWAYS;
                JScrollPane jsp=new JScrollPane(jt,v,h);
                c.add(jsp,BorderLayout.WEST);
        }
        public static void main(String args[])
        {
                JTreeDemo f1=new JTreeDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

JTable Class:

- JTable is a predefined class which is present under javax.swing package.
- JTable is a component which displays rows and column of data.
- If we want to display the data in tabular format, then we can use JTable class.
- *** **Procedure**:

1) Create JTable object with data and column heading
2) Create JScrollPane object and add JTable object in it
3) Add Scrollpane into the container.

Constructors:

1) JTable(Object data[][],Object ColHeads[])

Example:

```
import javax.swing.*;
import java.awt.*;
class JTableDemo extends JFrame
{
	JTableDemo()
	{
		Container c=getContentPane();
		String colheads[]={"RollNO","Name","Marks"};
		String data[][]={{"1010","Dennis","78.80"},
				{"2020","James","90.90"},
				{"3030","Peter","78.50"},
				{"4040","Yang","50.50"},
				{"5050","Brenden","80.45"}};
		JTable jt=new JTable(data,colheads);
		int v=ScrollPaneConstants.VERTICAL_SCROLLBAR_ALWAYS;
		int h=ScrollPaneConstants.HORIZONTAL_SCROLLBAR_ALWAYS;
		JScrollPane jsp=new JScrollPane(jt,v,h);
		c.add(jsp);
	}
	public static void main(String args[])
	{
		JTableDemo f1=new JTableDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("VJTech Frame");
	}
}
```

JProgressBar Class:

- JProgressBar is a predefined class which is present under javax.swing package.
- JProgressBar is a component that graphically shows the progress of task.
- In this progress bar, we can display the task completion percentage.

Constructors:

1) JProgressBar() - create horizontal progress bar.
2) JProgressBar(int orient) - orient => SwingConstants.VERTICAL, SwingConstants.HORIZONTAL
3) JProgressBar(int orient,int min,int max)
4) JProgressBar(BoundedRangeModel model)
5) JProgressBar(int min,int max)

Methods:

1) void setValue(int val)
2) void setStringPainted(boolean flag);

Example:

```
import javax.swing.*;
import java.awt.*;
class JProgressBarDemo extends JFrame
{
	JProgressBar jpb;
	JProgressBarDemo()
	{
		Container c=getContentPane();
		FlowLayout f1=new FlowLayout();
		c.setLayout(f1);
		jpb=new JProgressBar(0,3000);
		jpb.setStringPainted(true);
		c.add(jpb);
	}
	public void ChangeProgressBarValue()
	{
		int i=0;
		while(i<=3000)
		{
			jpb.setValue(i);
			i=i+20;
			try
			{
```

```
                                        Thread.sleep(100);
                                }
                                catch(Exception e)
                                {
                                        System.out.println(e);
                                }
                        }
                }
                public static void main(String args[])
                {
                        JProgressBarDemo f1=new JProgressBarDemo();
                        f1.setVisible(true);
                        f1.setSize(500,500);
                        f1.setTitle("VJTech Frame");
                        f1.ChangeProgressBarValue();
                }
}
```

JToolTip Class:

- It is used to display one pop-up message when your mouse pointer stays on a component.
- This pop-up message can be a short description or labelled information of any component.
- We can add tooltip text to almost all the components of Java Swing.
- To add tool tip to a component object. we have to use following method.

Method:

1) void setToolTipText(String str)

Example:

```
import javax.swing.*;
import java.awt.*;
class JToolTipDemo extends JFrame
{
	JToolTipDemo()
	{
		Container c=getContentPane();
		FlowLayout f1=new FlowLayout();
		c.setLayout(f1);
		JButton b1=new JButton("Login");
		b1.setToolTipText("This is login button");
		c.add(b1);
	}
	public static void main(String args[])
	{
		JToolTipDemo f1=new JToolTipDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("VJTech Frame");
	}
}
```

Introduction to Event Handling:

- Event handling is the mechanism that controls the interaction between a user and a GUI.
- It's how Java responds to actions like clicking a button, typing in a text field, or moving the mouse.

Basic Flow of Event Handling

1. **Event Source** – The object (like a button) that generates the event.
2. **Event Object** – An object that describes the event (like ActionEvent, MouseEvent).
3. **Event Listener** – An interface that receives the event and processes it.

Delegation Event Model:

- The **Delegation Event Model** in Java is a core design pattern used for **event handling**, especially in **GUI programming** with **AWT** and **Swing**.
- It enables a clean separation between event generation and event handling by delegating the task of handling an event to an external listener.
- The delegation event model defines the standard background mechanism to process on generated event.
- In this model, Source generates events and send that event to the event listener.

Overview of the Delegation Event Model

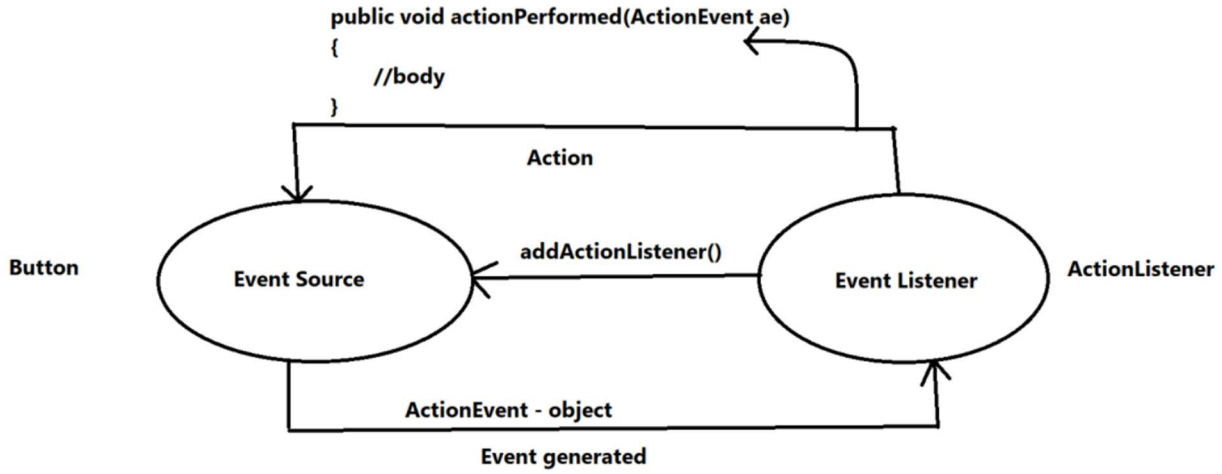


Figure:- Event Delegation Model

It involves **three main components**

1. **Event Object:** Event is an action performed on any source. Event is a results of user action which changes the state of source. Source will generate the event.
   Example: - clicking on push button, pressing key from keyboard, clicking mouse button, selecting item from list, check or uncheck checkbox, etc.

2. **Event Source**: Source is a component object on which action is performed. Source can generate more than one type of events. It is necessary that source must register listener. Each event has its own registration method.

   Example: Button, Choice, List, Checkbox, Menuitem, TextArea, TextField,etc

   We can use following method to register listener on source:

   Example: void addActionListener(), void addItemListener(), void addKeyListener(), void addWindowListener().

   We can remove the registration of listener using following methods:
   Example: removeActionListener(), removeItemListener(),removeWindowListener(), etc

3. **Event Listener** – Listener is an object which gets notified when event occurs. Listener listen the event which is generated by source and take action accordingly (called particular method).

   Example: MouseListener, ActionListener, WindowListener, ItemListener, etc

   We require to import java.awt.event package for event handling.

Class -> ActionEvent & Interface -> ActionListener:

❖ ActionEvent

- An ActionEvent class generates event when you press button, list item is double clicked, menu-item is selected.
- The ActionEvent class in Java has several constructors for creating instances of it.

Constructors:

1) ActionEvent(Object source, int id, String command)
2) ActionEvent(Object source, int id, String command, int modifiers)
3) ActionEvent(Object source, int id, String command, long when, int modifiers)

Where:

- **Source:** The Object that triggered the event (e.g, a button, a menu item).
- **ID:** An integer that identifies the type of event (e.g., ACTION_PERFORMED).
- **Command:** A string that provides additional information about the event (e.g., the text label of a button).
- **Modifiers:** Keys like Shift, Ctrl, Alt that were pressed along with the primary action.
- **Timestamp(when):** The time when the event occurred.

Methods:

1. String getActionCommand() - it will return label string of component.
2. Object getSource() – Returns the object that generated the event.
3. int getModifier() - it will return modifier value.
4. long getWhen() - it return time of event.

ActionListener: This is interface which defines following method that is invoked when an action event occurs.

```
public void actionPerformed(ActionEvent ae)
{
        //body of method
 }
```

Example:

```
import java.awt.*;
import java.awt.event.*;
class EventDemo extends Frame implements ActionListener
{
	EventDemo()
	{
		FlowLayout f1=new FlowLayout();
		setLayout(f1);
		Button b1=new Button("OK");
		b1.addActionListener(this);
		add(b1);
	}
	public void actionPerformed(ActionEvent ae)
	{
		System.out.println("You have pressed OK button");
	}
	public static void main(String args[])
	{
		EventDemo e1=new EventDemo();
		e1.setVisible(true);
		e1.setSize(500,500);
		e1.setTitle("VJTech Frame");
	}
}
```

Class -> ItemEvent & Interface -> ItemListener:

❖ ItemEvent

- ItemEvent class generates event when we select checkbox, when checkable menu item is selected, etc.
- There are two types of ItemEvent.

  1) DESELECTED - When user deselected an item.

  2) SELECTED - When user select an item.

- ItemEvent class defines one integer constant ITEM_STATE_CHANGED.

Constructors:

1. ItemEvent(ItemSelectable source, int id, Object item, int state)

Where:

- **Source:** The component (object) that fired the event.
- **ID:** The event type (usually `ItemEvent.ITEM_STATE_CHANGED`).
- **item:** The specific item that was changed.
- **state:** The new state of the item, either ItemEvent.SELECTED or ItemEvent.DESELECTED.

Methods:

1. Object getItem()
2. ItemSelectable getItemSelectable()
3. int getStateChange()

ItemListener: This is interface which defines following method that is invoked when an item event occurs.

```
public void itemStateChanged(ItemEvent ie)
{
        //body of method
}
```

Example:

```
import java.awt.*;
import java.awt.event.*;
class ItemEventDemo extends Frame implements ItemListener
{
	Checkbox c1,c2;
	Label L1;
	ItemEventDemo()
	{
		FlowLayout f1=new FlowLayout();
		setLayout(f1);
		c1=new Checkbox("India");
		c2=new Checkbox("America");
		L1=new Label("                                        ");
		add(c1);
		add(c2);
		add(L1);
		c1.addItemListener(this);
		c2.addItemListener(this);
	}
	public void itemStateChanged(ItemEvent ie)
	{
		String str="You have selected: ";
		if(c1.getState())
		{
			str=str+c1.getLabel()+" ";
		}
		if(c2.getState())
		{
			str=str+c2.getLabel()+" ";
		}
		L1.setText(str);
	}
	public static void main(String args[])
	{
		ItemEventDemo f1=new ItemEventDemo();
		f1.setVisible(true);
		f1.setSize(500,500);
		f1.setTitle("VJTech Frame");
	}
}
```

Class -> KeyEvent & Interface -> KeyListener:

KeyEvent

- When keyboard input is occurred then KeyEvent is generated.
- There are three different types of key event.

  1) KEY_PRESSED - when a key is pressed down.

  2) KEY_RELEASED - when a key is released.

  3) KEY_TYPED - when a key is typed

Constructors:

1. KeyEvent(Component source, int id, long when, int modifiers, int keyCode, char keyChar)

Where:

- **Source:** The component that fired the event.
- **ID:** The event type (`KEY_PRESSED`, `KEY_RELEASED`, `KEY_TYPED`).
- **when:** The timestamp.
- **modifies:** Modifier keys down during the event (e.g., Shift, Ctrl).
- **keyCode**: Integer code representing the key
- **keyChar:** The character associated with the key

Methods:

1. int getKeyCode() - Returns the integer key code
2. char getKeyChar() - Returns the character for the key typed
3. int getModifiers() - Returns the modifier keys used (e.g., Shift, Ctrl).

KeyListener: This is interface which defines following method that is invoked when an key event occurs.

```
public void keyPressed(KeyEvent ke)
{
        //body
}
public void keyTyped(KeyEvent ke)
{
        //body
```

```
}
public void keyReleased(KeyEvent ke)
{
        //body
}
```

Example:

```
import java.awt.*;
import java.awt.event.*;
class KeyEventDemo extends Frame implements KeyListener
{
        TextField tf1;
        Label L1,L2,L3;
        KeyEventDemo()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                tf1=new TextField(20);
                L1=new Label("                    ");
                L2=new Label("                    ");
                L3=new Label("                    ");
                add(tf1);
                add(L1);add(L2);add(L3);
                tf1.addKeyListener(this);
        }
        public void keyPressed(KeyEvent ke)
        {
                L1.setText("Key Pressed");
        }
        public void keyTyped(KeyEvent ke)
        {
                L2.setText("Key Typed");
        }
        public void keyReleased(KeyEvent ke)
        {
                L3.setText("Key Released");
        }
        public static void main(String args[])
        {
                KeyEventDemo f1=new KeyEventDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

Class -> MouseEvent & Interface -> MouseListener, MouseMotionListener

MouseEvent

- MouseEvent class will generates event when mouse buttons are pressed.
- Following are the integer constants related to mouse event .

1) MOUSE_CLICKED - when the user clicked the mouse
2) MOUSE_DRAGGED - When the user dragged the mouse.
3) MOUSE_ENTERED - When the mouse entered a component.
4) MOUSE_EXITED - When the mouse is exited from the component
5) MOUSE_MOVED - when the mouse moved
6) MOUSE_PRESSED - when the mouse was pressed.
7) MOUSE_RELEASED -When the mouse was released
8) MOUSE_WHEEL - When the mouse wheel was moved

Constructors:

1. MouseEvent(Component source, int id, long when, int modifiers, int x, int y, int clickCount, boolean popupTrigger)

Where:

- **Source**: The component that originated the event.
- **id**: The integer identifier for the event type (e.g., MouseEvent.MOUSE_CLICKED, MouseEvent.MOUSE_PRESSED, etc.).
- **when**: The time the event occurred (in milliseconds since the epoch).
- **modifiers**: The modifier keys down during event (e.g., InputEvent.SHIFT_DOWN_MASK).
- **x,y**: The x and y coordinates of the mouse event relative to the source component.
- **clickCount**: The number of mouse clicks associated with the event.
- **popupTrigger**: A boolean indicating whether this event is a trigger for a popup menu on the platform.

Methods:

1) int getX()
2) int getY()
3) int getClickCount()
4) boolean isPopupTrigger()

MouseListener: This is interface which defines following methods.

```
1) void mouseClicked(MouseEvent me) {//body}
2) void mouseEntered(MouseEvent me) {//body}
3) void mouseExited(MouseEvent me) {//body}
4) void mousePressed(MouseEvent me) {//body}
5) void mouseReleased(MouseEvent me) {//body}
```

MouseMotionListener: This is interface which defines following methods.

```
1) void mouseDragged(MouseEvent me) {//body}
2) void mouseMoved(MouseEvent me)  {//body}
```

Example-1:

```
//MouseListener implementation
import java.awt.*;
import java.awt.event.*;
class MouseEventDemo extends Frame implements MouseListener
{
	MouseEventDemo()
	{
		FlowLayout f1=new FlowLayout();
		setLayout(f1);
		Button b1=new Button("Login");
		add(b1);
		b1.addMouseListener(this);
	}
	public void mouseClicked(MouseEvent me)
	{
		System.out.println("Mouse Clicked");
	}
	public void mouseEntered(MouseEvent me)
	{
		System.out.println("Mouse Entered");
	}
	public void mouseExited(MouseEvent me)
	{
		System.out.println("Mouse Exited");
	}
	public void mousePressed(MouseEvent me)
	{
		System.out.println("Mouse Pressed");
```

```
        }
        public void mouseReleased(MouseEvent me)
        {
                System.out.println("Mouse Released");
        }
        public static void main(String args[])
        {
                MouseEventDemo f1=new MouseEventDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

Example-2:

```
//MouseMotionListener implementation
import java.awt.*;
import java.awt.event.*;
class MouseEventDemo1 extends Frame implements MouseMotionListener
{
        MouseEventDemo1()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                Button b1=new Button("Login");
                add(b1);
                addMouseMotionListener(this);
        }
        public void mouseDragged(MouseEvent me)
        {
                System.out.println("Mouse Dragged");
        }
        public void mouseMoved(MouseEvent me)
        {
                System.out.println("Mouse Moved");
        }
        public static void main(String args[])
        {
                MouseEventDemo1 f1=new MouseEventDemo1();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```

Class -> TextEvent & Interface -> TextListener

- TextEvent class generates event when user or program enter some characters in TextField or TextArea.
- The class TextEvent defines integer constant TEXT_VALUE_CHANGED

Constructors:

TextEvent(Object src, int type)

TextListener: This is interface which defines following method .

```
void textValueChanged(TextEvent te)
{
        //body
}
```

Example:

```
import java.awt.*;
import java.awt.event.*;
class TextEventDemo extends Frame implements TextListener
{
        TextField tf1;
        Label L1;
        TextEventDemo()
        {
                FlowLayout f1=new FlowLayout();
                setLayout(f1);
                tf1=new TextField(30);
                L1=new Label("                    ");
                add(tf1);add(L1);
                tf1.addTextListener(this);
        }
        public void textValueChanged(TextEvent te)
        {
                L1.setText("Text Value Changed");
        }
        public static void main(String args[])
        {
                TextEventDemo f1=new TextEventDemo();
                f1.setVisible(true);
                f1.setSize(500,500);
                f1.setTitle("VJTech Frame");
        }
}
```